

ChatGPT's Code:

```
#include <cstdint>
#include <fstream>
#include <iostream>
#include <iterator>
#include <list>
#include <sstream>
#include <string>
#include <utility>

class Email {
private:
    std::string sender;
    std::string subject;
    std::string date;
    int priority;
    int dateValue;

    static int senderPriority(const std::string& category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        return 1;
    }

public:
    Email(const std::string& senderCategory, const std::string&
subjectLine,
        const std::string& emailDate)
        : sender(senderCategory), subject(subjectLine), date(emailDate),
priority(senderPriority(senderCategory)),
dateValue(std::stoi(emailDate.substr(6, 4)) * 10000
          + std::stoi(emailDate.substr(0, 2)) * 100
          + std::stoi(emailDate.substr(3, 2))) {}

    bool higherPriorityThan(const Email& other) const {
        if (priority != other.priority) return priority > other.priority;
```

```

        return dateValue > other.dateValue;
    }

    void display(std::ostream& output) const {
        output << "Next email:\n"
                << "        Sender: " << sender << '\n'
                << "        Subject: " << subject << '\n'
                << "        Date: " << date << "\n\n";
    }
};

class MaxHeap {
private:
    // The list stores the heap in level order. Children of index i are
    // at 2*i + 1 and 2*i + 2; its parent is at (i - 1) / 2.
    std::list<Email> emails;

    Email& at(std::size_t index) {
        auto position = emails.begin();
        std::advance(position, static_cast<std::ptrdiff_t>(index));
        return *position;
    }

    void siftUp(std::size_t index) {
        while (index > 0) {
            std::size_t parent = (index - 1) / 2;
            Email& childEmail = at(index);
            Email& parentEmail = at(parent);
            if (!childEmail.higherPriorityThan(parentEmail)) break;
            std::swap(childEmail, parentEmail);
            index = parent;
        }
    }

    void siftDown(std::size_t index) {
        while (true) {
            std::size_t left = 2 * index + 1;
            if (left >= emails.size()) break;

            std::size_t right = left + 1;

```

```

        std::size_t highest = left;
        if (right < emails.size()
            && at(right).higherPriorityThan(at(left))) {
            highest = right;
        }

        Email& parentEmail = at(index);
        Email& childEmail = at(highest);
        if (!childEmail.higherPriorityThan(parentEmail)) break;
        std::swap(parentEmail, childEmail);
        index = highest;
    }
}

public:
    void insert(const Email& email) {
        emails.push_back(email);
        siftUp(emails.size() - 1);
    }

    const Email* peek() const {
        return emails.empty() ? nullptr : &emails.front();
    }

    void removeMax() {
        if (emails.empty()) return;
        if (emails.size() == 1) {
            emails.pop_front();
            return;
        }
        emails.front() = std::move(emails.back());
        emails.pop_back();
        siftDown(0);
    }

    std::size_t count() const {
        return emails.size();
    }
};

```

```

class EmailPrioritizer {
private:
    MaxHeap inbox;

    static std::string trim(const std::string& value) {
        std::size_t first = value.find_first_not_of(" \t\r\n");
        if (first == std::string::npos) return "";
        std::size_t last = value.find_last_not_of(" \t\r\n");
        return value.substr(first, last - first + 1);
    }

public:
    void run(std::istream& input, std::ostream& output) {
        std::string line;
        while (std::getline(input, line)) {
            std::istringstream record(line);
            std::string command;
            record >> command;

            if (command == "EMAIL") {
                std::string sender;
                std::string subject;
                std::string date;
                std::getline(record, sender, ',');
                std::getline(record, subject, ',');
                std::getline(record, date);
                inbox.insert(Email(trim(sender), trim(subject),
trim(date)));
            } else if (command == "NEXT") {
                const Email* next = inbox.peek();
                if (next != nullptr) {
                    next->display(output);
                } else {
                    output << "There are no emails to read.\n\n";
                }
            } else if (command == "READ") {
                inbox.removeMax();
            } else if (command == "COUNT") {
                output << "There are " << inbox.count()
                    << " emails to read.\n\n";
            }
        }
    }
};

```

```

    }
}

};

int main(int argc, char* argv[]) {
    EmailPrioritizer program;
    if (argc > 1) {
        std::ifstream input(argv[1]);
        if (!input) {
            std::cerr << "Unable to open test file.\n";
            return 1;
        }
        program.run(input, std::cout);
    } else {
        program.run(std::cin, std::cout);
    }
    return 0;
}

```

Gemini's Code:

```

#include <iostream>
#include <string>
#include <vector>
#include <sstream>

using namespace std;

class Email {
private:
    string sender;
    string subject;
    string date;
    int priorityScore;
    string comparableDate;

```

```

    int determinePriority(const string& s) {
        if (s.find("Boss") != string::npos) return 5;
        if (s.find("Subordinate") != string::npos) return 4;
        if (s.find("Peer") != string::npos) return 3;
        if (s.find("Important") != string::npos) return 2;
        return 1;
    }

    string parseDate(const string& d) {
        if (d.length() >= 10) {
            return d.substr(6, 4) + d.substr(0, 2) + d.substr(3, 2);
        }
        return d;
    }

public:
    Email(string s, string sub, string d) : sender(s), subject(sub),
    date(d) {
        priorityScore = determinePriority(sender);
        comparableDate = parseDate(date);
    }

    string getSender() const { return sender; }
    string getSubject() const { return subject; }
    string getDate() const { return date; }

    bool operator<(const Email& other) const {
        if (priorityScore != other.priorityScore) {
            return priorityScore < other.priorityScore;
        }
        return comparableDate < other.comparableDate;
    }
};

class MaxHeap {
private:
    vector<Email> heap;

    void heapifyUp(int index) {

```

```

        while (index > 0) {
            int parentIndex = (index - 1) / 2;
            if (heap[parentIndex] < heap[index]) {
                swap(heap[parentIndex], heap[index]);
                index = parentIndex;
            } else {
                break;
            }
        }
    }

    void heapifyDown(int index) {
        int size = heap.size();
        while (true) {
            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;
            int largest = index;

            if (leftChild < size && heap[largest] < heap[leftChild]) {
                largest = leftChild;
            }
            if (rightChild < size && heap[largest] < heap[rightChild]) {
                largest = rightChild;
            }

            if (largest != index) {
                swap(heap[index], heap[largest]);
                index = largest;
            } else {
                break;
            }
        }
    }
}

```

public:

```

    void insert(const Email& email) {
        heap.push_back(email);
        heapifyUp(heap.size() - 1);
    }

```

```

bool getNext(Email& outEmail) const {
    if (heap.empty()) return false;
    outEmail = heap[0];
    return true;
}

void readNext() {
    if (heap.empty()) return;
    heap[0] = heap.back();
    heap.pop_back();
    if (!heap.empty()) {
        heapifyDown(0);
    }
}

int getCount() const {
    return heap.size();
}
};

string trim(const string& str) {
    size_t first = str.find_first_not_of(" \t\r\n");
    if (string::npos == first) return "";
    size_t last = str.find_last_not_of(" \t\r\n");
    return str.substr(first, (last - first + 1));
}

int main() {
    MaxHeap emailQueue;
    string line;

    while (getline(cin, line)) {
        line = trim(line);
        if (line.empty()) continue;

        if (line.substr(0, 6) == "EMAIL ") {
            string data = line.substr(6);
            size_t pos1 = data.find(",");
            if (pos1 != string::npos) {

```

```

        size_t pos2 = data.find(",", pos1 + 1);
        if (pos2 != string::npos) {
            string sender = trim(data.substr(0, pos1));
            string subject = trim(data.substr(pos1 + 1, pos2 -
pos1 - 1));

            string dateStr = trim(data.substr(pos2 + 1));

            emailQueue.insert(Email(sender, subject, dateStr));
        }
    }
}

else if (line == "NEXT") {
    Email nextEmail("", "", "");
    if (emailQueue.getNext(nextEmail)) {
        cout << "Next email:" << "\n";
        cout << "Sender: " << nextEmail.getSender() << "\n";
        cout << "Subject: " << nextEmail.getSubject() << "\n";
        cout << "Date: " << nextEmail.getDate() << "\n";
    }
}

else if (line == "READ") {
    emailQueue.readNext();
}

else if (line == "COUNT") {
    cout << "There are " << emailQueue.getCount() << " emails to
read.\n";
}

}

return 0;
}

```

GenAI Analysis:

I utilized ChatGPT and Gemini to generate the two programs for this assignment. I accessed both of them through their websites. For the prompt, I

gave them a pdf and the instructions “Use these instructions to generate the program.” The pdf contained these instructions:

Assignment:

1. Use two GenAI’s to generate C++ code using objects, not functions, to implement the following requirements, which are the same as Assignment 2 (Note: You may use either ChatGPT or CoPilot as one of your choices, but not both. They are essentially the same GenAI.):
 - The program will prioritize emails for a busy company CEO.
 - You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
 - You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritizing for the CEO.
 - You must create objects from scratch. Do not include pre-existing heap modules.

• **Test File format:**

Command	Description
EMAIL <sender category>,<subject line>,<date>	The emails in the CEO’s Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

• **Sample Test File:**

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

• **Your program should display this:**

```
There are 6 emails to read.

Next email:
  Sender: Boss
  Subject: Never Mind
  Date: 01-03-2025

Next email:
  Sender: Boss
  Subject: Important
  Date: 12-20-2024

There are 4 emails to read.
```

In terms of correctness, both generated programs work as intended, however, ChatGPT used `std::list` while Gemini used a vector when the instructions stated it should use a “list-based” implementation. They both output the right text, and handle the errors mentioned such as repeated NEXT commands, READ commands without a NEXT, and commands on an empty inbox.

For execution time, Gemini's is a bit faster because the heap is stored in a vector. Adding and removing emails takes $O(\log n)$ time. ChatGPT's program uses a list, so adding or removing emails can be $O(n)$ time. In terms of space complexity, they are both the same with an $O(n)$ space. The space grows when the number of emails grows. Gemini used a vector while ChatGPT opted for a list.

ChatGPT's program would be easier to maintain because of how it split the functions of the program into clear labeled classes. This is more readable compared to Gemini's code which is less organized with more bulk in the main function. Both of the GenAIs did not include comments.

I chose to use ChatGPT for this assignment because as mentioned before, it is easier to maintain with the classes, and it utilized a proper list as the instructions requested.

To improve on ChatGPT's code, ChatGPT suggested that the list-based heap could be more efficient and correct based on the desired instructions. The current code moves through the list one at a time, which can be inefficient in a worst case scenario. ChatGPT suggested links between the heap nodes to fix this. This way adding or removing emails from the inbox would be $O(\log n)$ time. Space complexity would remain the same as $O(n)$. For maintainability, I added comments throughout the code describing what the lines or code blocks were doing.